

UNIVERSIDAD AUTÓNOMA DE CHIAPAS

Facultad de Contaduría y Administración C-1
Ingeniería en Desarrollo y Tecnologías de Software

Subsistema de notificaciones por correo en arquitectura hexagonal

Puerto de salida **EmailServicePort** y adaptador **NodemailerAdapter**
para el Sistema de Comercio Electrónico 5M

REPOSITORIO DEL CÓDIGO FUENTE

github.com/LuisAGA-1/ecommerce-5m

Alumno: **Luis Angel Alegria Gomez**

Semestre y grupo: 5.º semestre · Grupo M

Entorno de pruebas: Local (localhost) · WSL2 Ubuntu 24.04

Fecha: 8 de octubre de 2026

Lugar: Tuxtla Gutiérrez, Chiapas

1. Objetivo y alcance

Extender la arquitectura hexagonal del e-commerce agregando un **puerto de salida** y un **adaptador de salida** que automatizan el envío del comprobante de compra e instrucciones de pago por correo electrónico al generar una nueva orden.

El sistema ya no procesa pagos con tarjeta en línea: el pedido se registra con estado **Pendiente de pago** (en la base de datos se conserva el valor PENDIENTE) y se disparan dos correos:

- **Cliente:** desglose de la compra, total, folio y datos bancarios (banco, titular, cuenta, CLABE, referencia y plazo).
- **Administrador:** aviso de nuevo pedido con los datos del cliente y el monto.

El núcleo (dominio y aplicación) no importa Nodemailer ni lee variables de entorno: solo conoce el contrato `EmailServicePort`. Cambiar de proveedor (Ethereal, Mailtrap, SMTP real, SendGrid...) consiste en escribir otro adaptador y conectarlo en `server.js`.

2. Entregable 1 – Diagrama de arquitectura actualizado

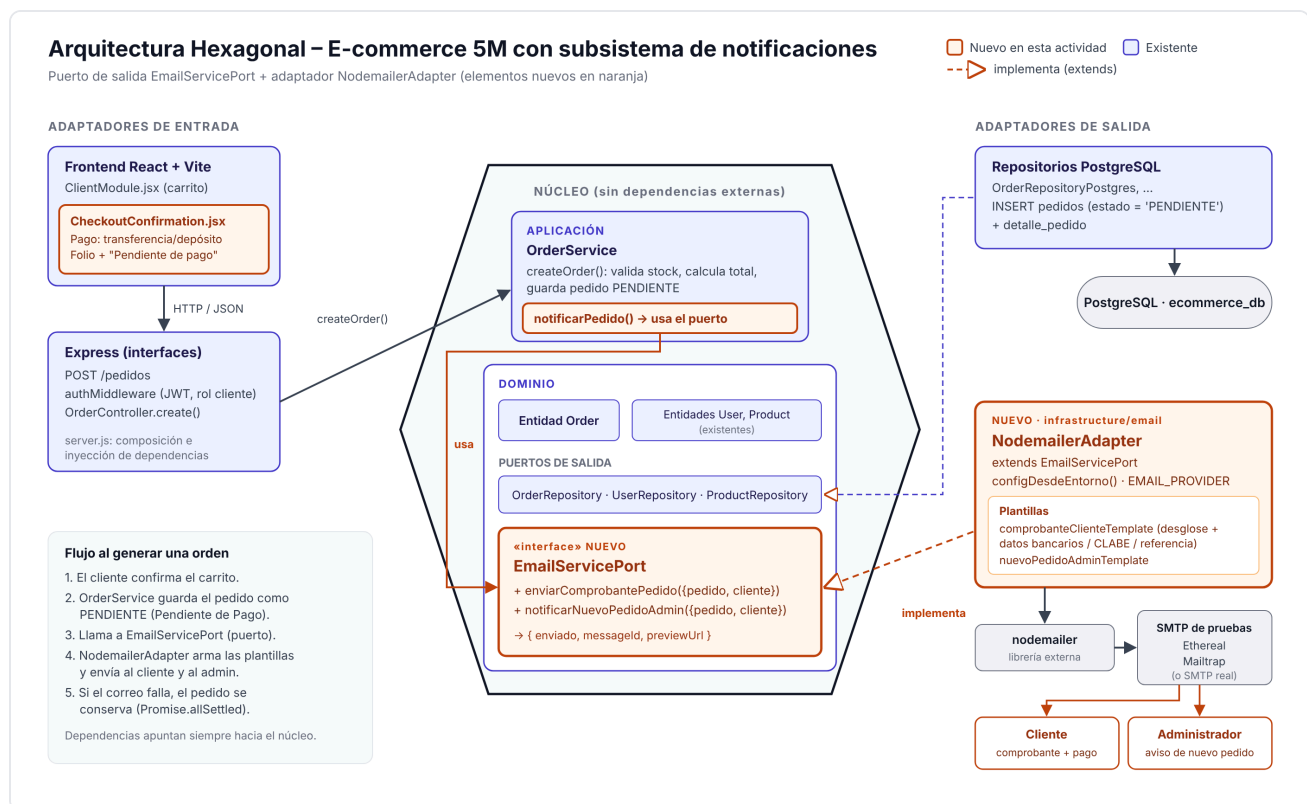


Figura 1. Arquitectura hexagonal con el nuevo puerto `EmailServicePort` y su adaptador `NodemailerAdapter` (en naranja). Archivo: docs/arquitectura-notificaciones.svg

Las dependencias apuntan hacia el núcleo: `OrderService` *usa* el puerto, y `NodemailerAdapter` *implementa* el puerto. La composición se hace en `server.js` mediante inyección por constructor, igual que con los repositorios existentes.

3. Entregable 2 – Código fuente backend y frontend

Todo el código está en el repositorio <https://github.com/LuisAGA-1/ecommerce-5m>. Archivos agregados y modificados:

Archivo	Capa	Tipo	Descripción
backend/src/domain/ports/EmailServicePort.js	Dominio	Nuevo	Puerto de salida (contrato) con enviarComprobantePedido y notificarNuevoPedidoAdmin.
backend/src/infraestructura/email/NodemailerAdapter.js	Infraestructura	Nuevo	Adaptador que implementa el puerto con Nodemailer (Ethereal, Mailtrap o SMTP).
backend/src/infraestructura/email/templates/comprobanteClienteTemplate.js	Infraestructura	Nuevo	Plantilla del cliente: desglose, total, datos bancarios, CLABE y referencia.
backend/src/infraestructura/email/templates/nuevoPedidoAdminTemplate.js	Infraestructura	Nuevo	Plantilla de aviso al administrador.
backend/src/infraestructura/email/templates/helpers.js	Infraestructura	Nuevo	Formato de moneda/fecha, folio, escape de HTML y layout.
backend/src/application/orders/OrderService.js	Aplicación	Modificado	Recibe el puerto por constructor y notifica tras crear el pedido.
backend/src/interfaces/server.js	Interfaces	Modificado	Composición: conecta NodemailerAdapter a OrderService.
backend/tests/notificaciones.test.js	Pruebas	Nuevo	6 pruebas con un adaptador falso en memoria (sin SMTP ni BD).
backend/scripts/probarCorreo.js	Script	Nuevo	npm run email:prueba: envía un pedido de ejemplo.
frontend/src/components/CheckoutConfirmation.jsx	Frontend	Nuevo	Confirmación del pedido: folio, Pendiente de pago, aviso de correo.
frontend/src/modules/client/ClientModule.jsx	Frontend	Modificado	Checkout con método de pago por transferencia y estados con etiqueta.
frontend/src/App.css	Frontend	Modificado	Estilos del checkout y la confirmación.
.env.example · package.json · README.md	Config	Nuevo / Mod.	Variables SMTP y de pago, dependencia nodemailer, scripts y documentación.

3.1 Puerto de salida (dominio)

```
// =====
// PUERTO DE SALIDA: EmailServicePort
//
// Define el contrato que el núcleo (dominio + aplicación)
// necesita para notificar por correo electrónico.
//
// El núcleo SOLO conoce este contrato. No sabe si el correo
// se envía con Nodemailer, SendGrid, Amazon SES, etc.
// Cualquier adaptador de infraestructura que extienda esta
// clase e implemente sus métodos puede conectarse al sistema
// sin modificar una sola línea de OrderService.
// =====
class EmailServicePort {

  /**
   * Envía al cliente el comprobante de su pedido junto con
   * las instrucciones para realizar el pago.
   *
   * @param {Object} params
   * @param {Object} params.pedido Pedido creado (id, fecha, total, estado, detalles[])
   * @param {Object} params.cliente Datos del cliente (id, nombre, email)
   * @returns {Promise<{ enviado: boolean, messageId?: string, previewUrl?: string }>}
   */
  async enviarComprobantePedido({ pedido, cliente }) {
    throw new Error(
      "Método enviarComprobantePedido no implementado"
    );
  }

  /**
   * Notifica al administrador que llegó un nuevo pedido.
   *
   * @param {Object} params
   * @param {Object} params.pedido
   * @param {Object} params.cliente
   * @returns {Promise<{ enviado: boolean, messageId?: string, previewUrl?: string }>}
   */
  async notificarNuevoPedidoAdmin({ pedido, cliente }) {
    throw new Error(
      "Método notificarNuevoPedidoAdmin no implementado"
    );
  }
}

module.exports = EmailServicePort;
```

3.2 Adaptador de salida (infraestructura)

Única clase que importa nodemailer. Selecciona el proveedor con EMAIL_PROVIDER; con Ethereum sin credenciales crea una cuenta de prueba automáticamente y devuelve la URL de vista previa.

```
class NodemailerAdapter extends EmailServicePort {
  ...
  // -----
  // Implementación del puerto
  // -----
  async enviarComprobantePedido({ pedido, cliente }) {

    const correo = comprobanteClienteTemplate({
      pedido,
      cliente,
      pago: this.config.pago,
      tienda: this.config.tienda
    });

    return this.enviar({ para: cliente.email, ...correo });
  }

  async notificarNuevoPedidoAdmin({ pedido, cliente }) {

    const correo = nuevoPedidoAdminTemplate({
      pedido,
      cliente,
      tienda: this.config.tienda,
      urlPanel: this.config.urlPanel
    });

    return this.enviar({ para: this.config.adminEmail, ...correo });
  }
}
```

3.3 Caso de uso: OrderService

El puerto se inyecta como cuarto parámetro opcional. Se usa `Promise.allSettled` para que un fallo de correo no revierta el pedido ya guardado.

```
// =====
// NOTIFICAR PEDIDO (usa el puerto EmailServicePort)
//
// El núcleo solo invoca métodos del puerto; no sabe qué
// librería ni qué proveedor envía el correo. Un fallo en el
// envío NO revierte el pedido: se registra y se informa.
// =====
async notificarPedido(pedido, cliente) {

  if (!this.emailService) {
    return {
      cliente: { enviado: false, motivo: "Servicio de correo no configurado" },
      admin: { enviado: false, motivo: "Servicio de correo no configurado" }
    };
  }

  const [cliRes, admRes] = await Promise.allSettled([
    this.emailService.enviarComprobantePedido({ pedido, cliente }),
    this.emailService.notificarNuevoPedidoAdmin({ pedido, cliente })
  ]);

  const resultado = r =>
    r.status === "fulfilled"
      ? r.value
      : { enviado: false, motivo: r.reason?.message || "Error al enviar" };

  if (cliRes.status === "rejected") {
    console.error("[pedido] No se envió el comprobante al cliente:", cliRes.reason);
  }

  if (admRes.status === "rejected") {
    console.error("[pedido] No se notificó al administrador:", admRes.reason);
  }

  return {
    cliente: resultado(cliRes),
    admin: resultado(admRes)
  };
}
```

```
// El repositorio crea el pedido como PENDIENTE (de pago).
const pedidoCreado =
  await this.orderRepository.create(order);

// Disparar notificaciones por el puerto de salida.
const notificacion = await this.notificarPedido(
  pedidoCreado,
  { id: user.id, nombre: user.nombre, email: user.email }
);

return { ...pedidoCreado, notificacion };
```

3.4 Composición en server.js

```
const NodemailerAdapter = require("../infrastructure/email/NodemailerAdapter");

// Aquí (y solo aquí) se decide qué implementación del puerto se conecta.
const emailService =
  (process.env.EMAIL_PROVIDER || "ethereal").toLowerCase() === "none"
    ? null
    : new NodemailerAdapter(NodemailerAdapter.configDesdeEntorno());

const orderService = new OrderService(
  orderRepository, userRepository, productRepository, emailService
);
```

3.5 Plantillas de correo

Funciones puras que reciben el pedido y devuelven { subject, html, text }. Generan versión HTML (con estilos en línea, compatibles con clientes de correo) y versión de texto plano. Todo dato del usuario se escapa para evitar inyección de HTML.



Figura 2. Plantilla del cliente (render local)

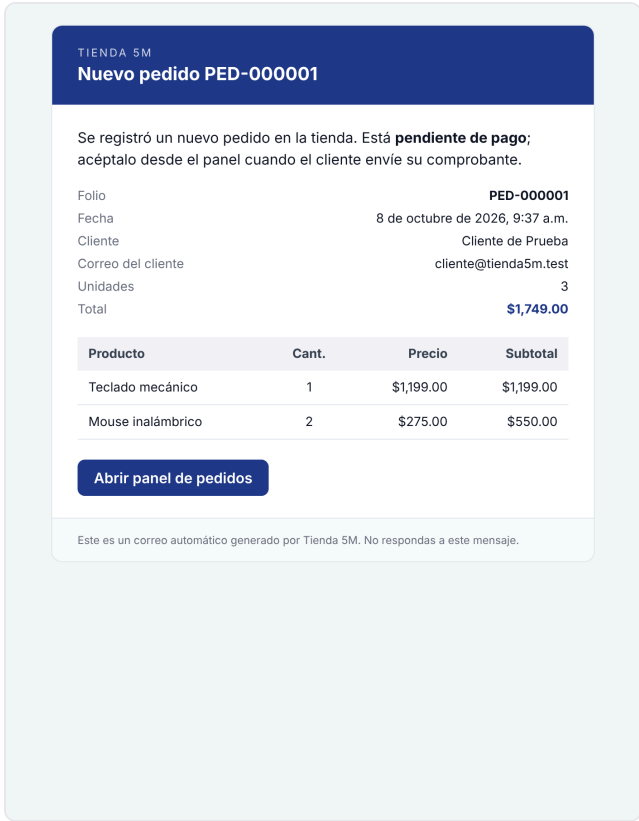


Figura 3. Plantilla del administrador (render local)

3.6 Checkout en React

El resumen del carrito muestra el método de pago **Transferencia o depósito bancario** y el botón **Confirmar pedido**. Al crearse la orden se muestra CheckoutConfirmation con el folio, la etiqueta *Pendiente de pago*, el desglose, los siguientes pasos y si el correo se envió. En modo desarrollo aparecen enlaces a la vista previa de Ethereum. En "Mis pedidos" el estado PENDIENTE se muestra como *Pendiente de pago*.

```
const pedidoCreado = await createOrder({ usuarioId: usuario.id, detalles });
setCarrito([]);
setPedidoConfirmado({ ...pedidoCreado, emailCliente: usuario.email });

// En la vista "carrito":
{pedidoConfirmado ? (
  <CheckoutConfirmation
    pedido={pedidoConfirmado} // folio, detalles, total, notificacion
    formatearPrecio={formatearPrecio}
    onVerPedidos={() => cambiarVista("pedidos")}
    onSeguirComprando={() => cambiarVista("productos")}
  />
) : /* carrito + método de pago "Transferencia o depósito" */ }
```

4. Entregable 3 – Evidencia de pruebas de envío

4.1 Pruebas unitarias del puerto (npm test)

Se conecta un adaptador falso en memoria a OrderService, sin Nodemailer, SMTP ni base de datos; esto demuestra el desacoplamiento del núcleo.

```
ok 1 - al crear un pedido se envía comprobante al cliente y aviso al admin
ok 2 - si el correo falla, el pedido se registra de todos modos
ok 3 - sin adaptador de correo el pedido se crea sin notificar
ok 4 - un adaptador que no implementa el puerto lanza error
ok 5 - la plantilla del cliente incluye folio, desglose, total y datos bancarios
ok 6 - la plantilla del administrador incluye cliente y total
# tests 6
# pass 6
# fail 0
```

4.2 Bandeja de pruebas (Ethereal / Mailtrap)

Pedido generado desde <http://localhost:5173> con un usuario cliente; los correos llegan a la bandeja de pruebas:

Captura 1

Checkout: confirmación del pedido

Pantalla de React con el folio y la etiqueta "Pendiente de pago".

Captura 2

Bandeja de Ethereal/Mailtrap – correo del cliente

Comprobante con desglose, total e instrucciones de pago del pedido generado.

Captura 3

Bandeja de Ethereum/Mailtrap – correo del administrador

Aviso de nuevo pedido con datos del cliente y total.

Captura 4

Consola del backend

Líneas [email] con la URL de vista previa de cada mensaje.

5. Ejecución local (localhost)

```
# 1. Obtener el código
git clone https://github.com/LuisAGA-1/ecommerce-5m.git
cd ecommerce-5m
npm install # instala nodemailer

# 2. Configurar variables
cp .env.example .env # BD + EMAIL_PROVIDER=ethereal (o mailtrap)

# 3. Probar
npm test # pruebas del puerto (sin BD ni SMTP)
npm run email:prueba # envía un correo de ejemplo

# 4. Levantar el sistema
npm run dev # backend -> http://localhost:3000
cd frontend && npm install && npm run dev # frontend -> http://localhost:5173
```

Proveedor	Configuración	Dónde ver los correos
Ethereal	EMAIL_PROVIDER=ethereal; sin SMTP_USER/SMTP_PASS se crea una cuenta al arrancar.	URL de vista previa en la consola, o iniciar sesión en ethereal.email con las credenciales impresas.
Mailtrap	EMAIL_PROVIDER=mailtrap + SMTP_USER y SMTP_PASS del Sandbox.	Email Testing → Inboxes en mailtrap.io .
Ninguno	EMAIL_PROVIDER=none	El pedido se crea sin enviar correos.

6. Conclusión

El subsistema de notificaciones se integró sin modificar la lógica de dominio del pedido: OrderService depende únicamente de la abstracción EmailServicePort, mientras que los detalles de Nodemailer, el proveedor SMTP y el contenido de las plantillas quedan en la capa de infraestructura. Las pruebas con un adaptador en memoria confirman que el núcleo funciona sin la librería externa y que un fallo de correo no afecta el registro del pedido, que queda *Pendiente de pago* hasta que el administrador valida la transferencia.

CÓDIGO FUENTE

<https://github.com/LuisAGA-1/ecommerce-5m>